



نظري

2
12
2340 sp

كلية الهندسة المعلوماتية

السنة الثانية

Type inference and Understanding Static

د. غاندي هسام



RB Informatics; 23/10/2025

برمجة ٣

VAR

مفهوم الـ **var** : في الإصدارات القديمة من الـ **JDK** وخصوصاً الإصدارات قبل الـ **JDK 10** عند كتابة الكود كان يجب الإعلان عن جميع المتغيرات المحلية بشكل صريح وتحديد نوعها قبل استخدامها .

مثال:

```
int x;  
String name;
```

وعند تهيئة المتغير كان يجب ان يكون نوع القيمة المسندة مطابقة للنوع المعلن .

```
int x=10;
```

لكن بدءاً من الإصدار **JDK 10** أصبح من الممكن للمترجم أن يستنتج نوع المتغير المحلي بناءً على نوع القيمة المسندة إليه باستخدام الكلمة المفتاحية **var** بهدف تبسيط الكود وجعله أنظف وتقليل الكلمات المتكررة

مثال:

`var message="hello world";` → جافا تستنتج أنه من النوع `String`

الشروط الأساسية لاستخدام var :

1. استخدام **var** كنوع : للإعلان عن المتغير يجب استخدام الكلمة المفتاحية **var** بدلاً من اسم النوع الحقيقي (int ,float,...)

```
double avg =10.0;
```

↓
نستبدل اسم النوع الحقيقي بـ **var**

```
var avg =10.0;
```

2. ضرورة التهيئة : يجب أن يتم تهيئة المتغير فوراً في نفس السطر لكي يتمكن المترجم من

استنتاج النوع مثال : `var avg=10.0;`

■ ملاحظة:

`var y ;` ❌

يجب ان نهين المتغير y اولاً وإلا سوف يحدث خطأ في الترجمة (`compile error`) .

■ 3. الثبات في النوع : لمجرد استنتاج النوع لا يمكن تغيير نوع المتغير لاحقاً .

■ مثال:

`var number=100 ;`

المترجم يستنتج نوع المتغير number أنه `int`

`number ="twenty";`

يحدث خطأ لا يمكن إسناد `String` الى متغير النوع `int`

■ ملاحظة:

عملية استنتاج النوع تحدث في وقت الترجمة `compile time` وليس في وقت التشغيل

`runtime`

■ مثال code :

```
1 public class VarDemo {
2
3
4     public static void main(String[] args) {
5         var avg=10.0;
6         System.out.println("value of avg:" +
avg );
7         int var=1;
8         System.out.println("value of var: "+var
);
9         var k= -var ;
10        System.out.println("value of k :"+ k);
11    }
12 }
```



■ شرح ال code:

السطرين 5 و 6 : يتم استخدام **var** للإعلان عن المتغير المحلي **avg** حيث يستنتج المترجم نوعه من قيمة التهيئة وبالتالي يصبح **avg** من النوع **double**.

السطرين 7 و 8 : يوضحان أن **var** ليست كلمة محجوزة بالكامل في **java** وحساسة للسياق أي خارج سياق للإعلان عن المتغير محلي يمكن استخدام **var** كأسم المتغير العادي.

السطرين 9 و 10 :

هنا يتم استخدام **var** كأسم المتغير الذي تم تعريفه سابقاً
int var=1;
 هنا تعمل **var** كأسم نوع والمتبرم يستنتج نوع المتغير **k**
var k= -var;
 قيمة **k** ستكون ← **k=-1**

الخرج:

Value of avg: 100
 Value of var: 1
 Value of k: -1

var في ال array

• في الفقرة السابقة استخدمنا **var** لإعلان عن متغيرات بسيطة فقط ولكن يمكنك أيضاً استخدام **var** لإعلان

عن مصفوفة **Array** بشروط 😊

■ لا يسمح بوضع المصفوفة **[]** بجوار **var** او اسم المتغير في الجانب أقواس

■ يجب أن يتم استنتاج النوع الكامل للمصفوفة من قيمة التهيئة

var []myArray = new int[10]; ❌

var myArray []=new int [10]; ❌

■ الطريقة الصحيحة:

var my Array =new int [10]; ✓

⬅ الأفكار السابقة قدمت أساسيات استدلال النوع للمتغيرات المحلية باستخدام الأنواع الأولية **primitive types** ولأن سننتقل للاستدلال على نوع المتغيرات المحلية من أنواع المراجع.

Reference Type

- ماهي أنواع المراجع reference type ؟ هي الأنواع التي تشير إلى كائنات في الذاكرة
- كل فئة (class) في جافا هي نوع مرجعي مثل الفئات المدمجة Built_in_class (String , ArrayList, Scanner) userDefined classes

الإعلان بالطريقة التقليدية

- ArrayList <String> myList = new ArrayList<String>();
- String name="Farah";
- هنا تظهر فائدة ال var عندما يكون اسم النوع طويلاً أو معقد تسمح لنا بتجنب تكرار كتابته
- Var myList =new ArrayList <String>();
- Var name ="Farah";
- حيث وقّرت هنا كتابة التعبير الطويل مرتين مما يجعل الكود أقصر وأسهل قراءة وأقل عرضة للأخطاء مع الحفاظ على النوع الحقيقي للمتغير

مثال بـ code :

```

1 public class MyClass {
2
3     private int i;
4
5     public MyClass(int k) {
6         i = k;
7     }
8
9
10    public int geti() {
11        return i;
12    }
13

```



**Big Dreams Have
Small Beginning.**

```
14 public void seti(int k) {
15     if (k >= 0) {
16         i = k;
17     }
18 }
19
20 }
21
22 public class VarDemo2 {
23
24     public static void main(String args[]) {
25         var mc = new MyClass(10);
26         System.out.println("value of i in mc is " + mc.geti());
27         mc.seti(19);
28         System.out.println("value of i in mc is now " + mc.geti());
29
30     }
31 }
```

- هذا المثال يوضح كيفية استخدام ميزة استدلال النوع للمتغيرات المحلية مع فئة معرفة من قبل المستخدم
 - في التابع الرئيسي *main* : نلاحظ أن عند إنشاء *object* تم استخدام *var* بدلاً من اسم الكلاس *MyClass*
 - المترجم يستنتج أن *mc* هو كائن من النوع *MyClass* لأن قيمة التهيئة هي *new MyClass(10)*
 - نلاحظ أيضاً أنه لا يوجد أي تغيير في سلوك الدالة على الرغم من استخدام *var* فإن *mc* يتصرف تماماً ككائن من النوع *MyClass* نستطيع الوصول إلى جميع دوال الفئة
- الخرج :

- Value of i in mc is 10
- value of i in mc is now 19

- مع تقدمك في دراسة *java* سنجد أن العديد من أنواع الفئات لها أسماء طويلة نسبياً مثل *FileInputStream* التي تستخدم لفتح ملف عمليات الإدخال في الطريقة التقليدية يتم تكرار اسم الفئة الطويلة مرتين في سطر الإعلان

```
FileInputStream fin =new FileInputStream("test.txt");
```

باستخدام *var* يمكن كتابة السطر بشكل أكثر إيجاز وتبسيط (stream line)

```
var fin =new FileInputStream("test.txt");
```

Using Local Variable Type Inference in for Loop

- يمكن استخدام **var** لتحديد المتغيرات التحكم والتكرار في حلقات ال **for** مثال :

```
for(var x=2.5 ; x<100 ; x*2 );
```

هنا تم استخدام **var** لإعلان عن المتغير **x**

- بما أن القيمة المسندة ل **x** هي 2.5 وهي قيمة **double** فإن المترجم يستنتج أن **x** من النوع **double** هذا الاستخدام يقلل من تكرار كتابة النوع **double** في سطر الحلقة **for**
- مع متغير التكرار في حلقة **foreach** المخصصة للتكرار على مصفوفة أو مجموعة

```
int []nums = {1,2,3,4,5,6};
```

 يتم تعريف مصفوفة من الاعداد الصحيحة

```
for(var v:nums)
```

هنا تم استخدام **var** بدلاً من **int** للإعلان عن متغير التكرار

بما أن المصفوفة **nums** تحتوي على عناصر نوع **int** فإن المترجم يستنتج أن متغير التكرار **v** هو أيضاً من النوع **int**



Keep

Going..

الحالات التي لا يمكن فيها استخدام **var**

- يمكن الإعلان عن متغير واحد فقط في كل مرة أي لايسمح بإعلان متغيرات متعددة في سطر واحد باستخدام **var**
 مثال: **var x=1,y=2;** ✗
- لا يمكن للمتغير استخدام **null** كقيمة تهيئة يجب أن تكون قيمة التهيئة لها نوع محدد **int**, **String**
 لايمكن للمترجم استنتاج نوع محدد من القيمة **var obj = null;** ✗
- null** لايمكن استخدام **var** مع صيغة تهيئة لمصفوفة المختصرة لأن هذه الصيغة لاتوفر معلومات كافية للمترجم لاستنتاج النوع يجب دائماً استخدام **new** لتحديد نوع المصفوفة

أمثلة:

```
var myArray = {1,2,3};
```

✗

هنا لا يعرف المترجم إذا كانت المصفوفة من `int[]` اعداد صحيحة أو أعداد صحيحة ككائنات .

```
var myArray = new int [10];
```

يرى المترجم كلمة `new int [10]` ويفهم مباشرة نوع المصفوفة وهو `int`

ملاحظات إضافية:

1. لانستطيع استخدام `var` كأحد حقول الكلاس

```
public class Demo {
var avg = 10.0 ;
}
```

✗ Error

2. أو كمعاملات الدوال

```
public void Print(var message)
```

3. أو القيم مرجعة من الدوال

```
public var sum(){
return 1;
}
```

❖ في ختام فقرة مفهوم ال Var :

إن الكلمة المفتاحية `var` هي مجرد صياغة نحوية (شكلية) لا تؤثر بأي شكل من الأشكال على عملية التنفيذ أو على استخدام الذاكرة

وإنما وجدت لتسهيل كتابة الكود وجعله أنظف وأكثر اختصاراً دون الحاجة إلى تكرار الأنواع في كل مرة. فبدلاً من تحديد نوع المتغير أو المصفوفة يدوياً منذ البداية يتولى المترجم استنتاج النوع تلقائياً من خلال استخدام `var`.

أي أنها ليست سوى أسلوب مختصر في الكتابة لا يغير شيئاً في زمن التنفيذ أو في استهلاك الذاكرة...



Static

- مفهوم ال static: في سياق البرمجة غرضية التوجه oop يعد مفهوم ال static في حال كنت ترغب بإنشاء object من كلاس بحيث يتم استخدامه بشكل مستقل عن أي كائن من تلك الفئة (كلاس).
- حيث توفر الكلمة المفتاحية ال static آلية لتعريف متغيرات ودوال يمكن استخدامها والوصول إليه بشكل مستقل تماماً عن أي كائن منشأ من تلك الفئة هذا يسمح بوجود بيانات مشتركة بين جميع الكائنات لا ترتبط بحالة كائن معين مما يمنحنا مرونة أكبر في تنظيم بنية البرنامج .

طريقة الإنشاء :

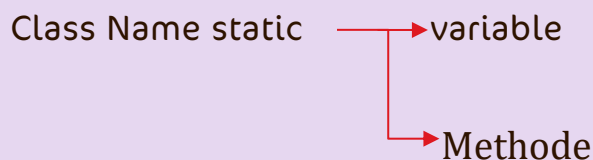
يتم وضع الكلمة المفتاحية static قبل الإعلان عن متغير أو دلالة .

المثال الأبرز :

الدالة `main()` هي المثال الأكثر شيوعاً لدالة static وهذا هو السبب في أة برنامج java يبدأ بالعمل دون الحاجة لإنشاء كائن من الفئة الرئيسية.

كيفية الوصول للمتغيرات أو الدوال الثانية :

يمكن الوصول إليه باستخدام اسم الفئة (class) مباشرة متبوعاً بمعامل النقطة (.) ثم اسم العنصر



على سبيل المثال إذا كنت تريد إسناد القيمة 10 الى متغير ثابت يسمى count وهو جزء من الفئة Timer (class) نكتب

```
Timer.count = 10;
```

يمكن استدعاء الدالة الثابتة (static methode) بنفس الطريقة

لايوجد حاجة لإنشاء للكائن object من الفئة للوصول الى الدوال أو المتغير

سلوك المتغيرات الثابتة

- المتغيرات المعلنه ك static هي بشكل أساسي متغيرات عامة `global variables` عند الإعلان عن الكائن لا يتم إنشاء أي نسخة من المتغير الثابت بدلاً من ذلك تشارك جميع نسخ الفئة في المتغير الثابت ذاته .

● مثال يوضح الفرق بين متغير عادي ومتغير ثابت :

```
1 class StaticDemo {
2
3     int x;
4     static int y;
5
6     int sum() {
7         return x + y;
8     }
9 }
10
11 class SDemo {
12     public static void main(String args[]) {
13         StaticDemo obj1 = new StaticDemo();
14         StaticDemo obj2 = new StaticDemo();
15
16         obj1.x = 10;
17         obj2.x = 20;
18
19         System.out.println("obj1.x = " + obj1.x);
20         System.out.println("obj2.x = " + obj2.x);
21
22         StaticDemo.y = 19;
23
24         System.out.println("obj1.sum() : " + obj1.sum());
25         System.out.println("obj2.sum() : " + obj2.sum());
26
27         StaticDemo.y = 100;
28
29         System.out.println("obj1.sum() : " + obj1.sum());
30         System.out.println("obj2.sum() : " + obj2.sum());
31     }
32 }
```



- السطر 3 : **instance variable** يتم إنشاء نسخة جديدة من هذا المتغير لكل كائن يتم إنشاؤه من الفئة StaticDemo.
- قيمة x لكائن معين تكون مستقلة عن قيمة x في كائن آخر .
- السطر 4 : **static variable** هناك نسخة واحدة فقط من هذا المتغير يتم تخزينها في الذاكرة ويتم مشاركتها بين جميع الكائنات المنشأة من الفئة StaticDemo إذا قام كائن بتغيير قيمة y فإن هذا التغيير سيظهر في جميع الكائنات الأخرى.
- السطر 6 و 7: هذه الدالة ترجع مجموع المتغير العادي x والمتغير الثابت y نلاحظ أن الدالة يمكنها الوصول إلى كلا النوعين من المتغيرات في الدالة الرئيسية **main** :
- 13,14: يتم إنشاء كائنين منفصلين obj2,obj1 من الفئة StaticDemo
- 16,17: يتم إسناد قيم مختلفة للمتغير x لكل كائن هذا ممكن لأن لكل كائن نسميه الخاصة المستقلة من x
- 19,20: طباعة قيم x
- $obj1=10$
 $obj2=20$ ← أي أنهما مستقلان
- 14: يتم إسناد القيمة 19 إلى المتغير الثابت y يتم الوصول إليه باستخدام اسم الفئة مباشرة دون الحاجة لاستخدام كائن
- 24,25: يتم طباعة نتيجة استدعاء الدالة sum() لكل كائن
- $obj1 \quad x+y=10+19=29$
- $obj2 \quad x+y=20+19=39$
- نلاحظ أن الكائنين يستخدمان نفس القيمة 19 للمتغير y
- 27: يتم تغيير قيمة المتغير y المشترك إلى 100 أيضاً باستخدام اسم الفئة
- 29,30: يتم تكرار عملية الطباعة
- $obj1 \quad x+y=10+100=110$
- $obj2 \quad x+y=20+100=120$
- نلاحظ أي تغيير في قيمة y من أي مكان ينعكس فوراً في حسابات جميع الكائنات .

استخدام الدوال الثابتة static methods

- الدوال الثابتة مصممة للعمل على متغيرات الثابتة فقط لا يمكنها الوصول بشكل مباشر إلى المتغيرات غير الثابتة

```

1 class StaticMeth {
2     static int val = 1024;
3     static int valDiv2() {
4         return val / 2;
5     }
6 }
7
8 class SDemo2 {
9     public static void main(String args[]) {
10        System.out.println("val is " + StaticMeth.val);
11        System.out.println("StaticMeth.valDiv2(): " + StaticMeth.valDiv2());
12
13        StaticMeth.val = 4;
14
15        System.out.println("val is " + StaticMeth.val);
16        System.out.println("StaticMeth.valDiv2(): " + StaticMeth.valDiv2());
17    }
18 }

```

■ كلاس StaticMeth :

■ 3,2: تم تعريف متغير ثابت

■ ودالة ثابتة تقوم بقسمة المتغير الثابت على 2

■ في الدالة main :

■ 11, 10: يتم الوصول للمتغير الثابت باستخدام اسم الفئة مباشرة ويطبع 1024

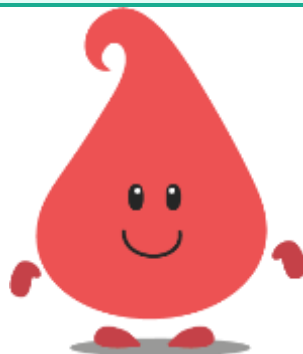
■ يتم استدعاء الدالة valDiv2() باستخدام اسم الفئة مباشرة ويطبع 1024/2=512

■ 13: تغيير قيمة المتغير الثابت

■ يتم تغيير قيمة المتغير val إلى 4

■ 15 : يطبع 4

■ 16 : يتم استدعاء الدالة الثابتة مرة أخرى ولكنها تعمل على القيمة الجديدة للمتغير val 4/2=2



قيود الدوال الثابتة

تنبع هذه القيود من طبيعتهما المستقلة عن الكائن

1. يمكن للدوال الثابتة استدعاء الدوال الثابتة الأخرى فقط داخل فئتها بشكل مباشر لأن الدوال العادية الغير ثابتة تتطلب كائن للعمل والدالة الثابتة لاتملك كائن .
2. يمكن للدوال الثابتة الوصول بشكل مباشر إلى المتغيرات الثابتة فقط لأن المتغيرات العادية لاوجود لها إلا بعد إنشاء كائن والدالة الثابتة تستدعي بدون كائن.
3. لا تمتلك مرجع `this` لأن المرجع `this` يشير إلى الكائن الحالي وبما أن الدالة الثابتة تستدعي دون كائن فلا يمكن أن يكون لديها مرجع `this` .

مثال:

```
1 public class Error {
2     int denom=3;
3     static int val=1024;
4     static int valDivDenom(){
5         return val/denom;
6     }
7 }
```

- السطر 2 : إنشاء متغير غير ثابت
- السطر 3 : إنشاء متغير ثابت
- السطر 4: دالة ثابته تحاول الوصول إلى `denom` وهذا غير صحيح
- لأنه متغير غير ثابت لذلك يظهر خطأ في التعريف



وتشاء أنت من الأمانى نجمة، ويشاء ربك أن يناولك القمر .